

1. Write a C program to simulate a Deterministic Finite Automata (DFA) for the given language representing strings that start with a and end with a

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char str[100];
```

```
    printf("Enter string: ");
```

```
    scanf("%s", str);
```

```
    int n = strlen(str);
```

```
    if (str[0] == 'a' && str[n - 1] == 'a')
```

```
        printf("Accepted\n");
```

```
    else
```

```
        printf("Rejected\n");
```

```
    return 0;
```

```
}
```

Output
Enter string: ni Rejected

2. Write a C program to simulate a Deterministic Finite Automata (DFA) for the given language representing strings that start with 0 and end with 1

```
#include <stdio.h>
```

```
#include <string.h>
```

```

int main() {
    char str[100];

    printf("Enter binary string: ");
    scanf("%s", str);

    int n = strlen(str);

    if (str[0] == '0' && str[n - 1] == '1')
        printf("Accepted\n");
    else
        printf("Rejected\n");

    return 0;
}

```

Output
<pre> Enter binary string: 01 Accepted </pre>

3. Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

$S \rightarrow 0A1$ $A \rightarrow 0A \mid 1A \mid \epsilon$

```
#include <stdio.h>
```

```
#include <string.h>
```

```

int main() {
    char str[100];

    printf("Enter string: ");

```

```

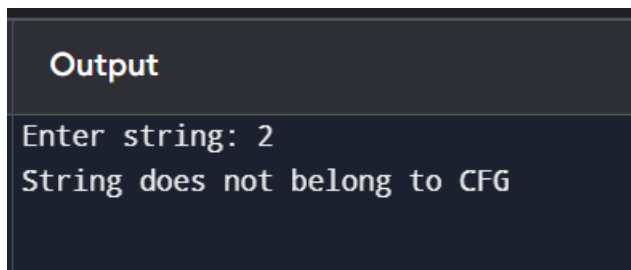
scanf("%s", str);

int n = strlen(str);

if (n >= 2 && str[0] == '0' && str[n - 1] == '1')
    printf("String belongs to CFG\n");
else
    printf("String does not belong to CFG\n");

return 0;
}

```



```

Output
Enter string: 2
String does not belong to CFG

```

4. Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

$S \rightarrow OS0 \mid 1S1 \mid 0 \mid 1 \mid \epsilon$

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char str[100];
```

```
    int i, n;
```

```
    printf("Enter string: ");
```

```
    scanf("%s", str);
```

```
    n = strlen(str);
```

```

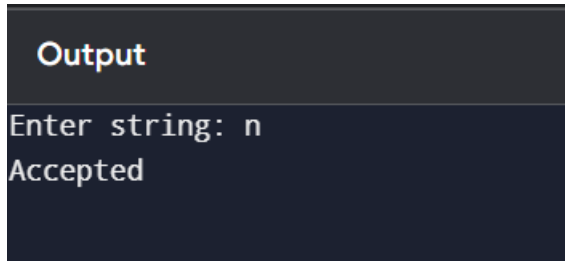
for (i = 0; i < n / 2; i++) {
    if (str[i] != str[n - 1 - i]) {
        printf("Not Accepted\n");
        return 0;
    }
}

```

```

printf("Accepted\n");
return 0;
}

```



```

Output
Enter string: n
Accepted

```

5. Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

$S \rightarrow OS0 \mid A \quad A \rightarrow 1A \mid \epsilon$

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char str[100];
```

```
    int n, i = 0, j;
```

```
    printf("Enter string: ");
```

```
    scanf("%s", str);
```

```
    n = strlen(str);
```

```
    j = n - 1;
```

```

while (i < j && str[i] == '0' && str[j] == '0') {
    i++;
    j--;
}

```

```

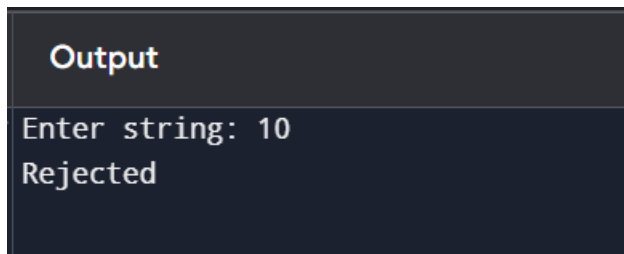
while (i <= j) {
    if (str[i] != '1') {
        printf("Rejected\n");
        return 0;
    }
    i++;
}

```

```

printf("Accepted\n");
return 0;
}

```



```

Output
Enter string: 10
Rejected

```

6. Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

$S \rightarrow OS1 \mid \epsilon$

```
#include <stdio.h>
```

```
#include <string.h>
```

```

int main() {
    char str[100];
    int n, i = 0;

```

```

printf("Enter string: ");
scanf("%s", str);

n = strlen(str);

while (str[i] == '0')
    i++;

int zeros = i;

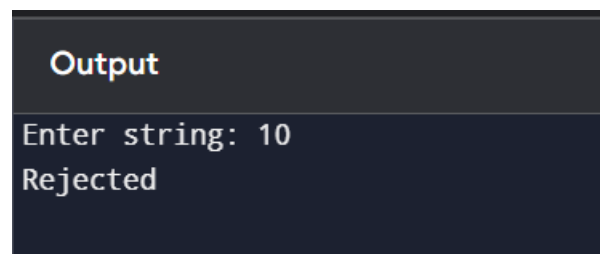
while (str[i] == '1')
    i++;

int ones = i - zeros;

if (i == n && zeros == ones)
    printf("Accepted\n");
else
    printf("Rejected\n");

return 0;
}

```



```

Output
Enter string: 10
Rejected

```

7. Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

$S \rightarrow A101A, A \rightarrow 0A \mid 1A \mid \epsilon$

```
#include <stdio.h>
```

```
#include <string.h>

int main() {
    char str[100];

    printf("Enter string: ");
    scanf("%s", str);

    if (strstr(str, "101") != NULL)
        printf("Accepted\n");
    else
        printf("Rejected\n");

    return 0;
}
```

Output
Enter string: 101 Accepted

8. Write a C program to simulate a Non-Deterministic Finite Automata (NFA) for the given language representing strings that start with b and end with a

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];

    printf("Enter string: ");
    scanf("%s", str);
```

```

int n = strlen(str);

if (str[0] == 'b' && str[n - 1] == 'a')
    printf("Accepted\n");
else
    printf("Rejected\n");

return 0;
}

```

Output
Enter string: a
Rejected

9. Write a C program to simulate a Non-Deterministic Finite Automata (NFA) for the given language representing strings that start with 0 and end with 1

```

#include <stdio.h>
#include <string.h>

int main() {
    char str[100];

    printf("Enter binary string: ");
    scanf("%s", str);

    int n = strlen(str);

    if (str[0] == '0' && str[n - 1] == '1')
        printf("Accepted\n");
    else
        printf("Rejected\n");
}

```



```
    return 0;
}
```

Output

```
Enter binary string: 01
Accepted
```

10. Write a C program to find ϵ -closure for all the states in a Non-Deterministic Finite Automata (NFA) with ϵ -moves.

```
#include <stdio.h>
```

```
int main() {
    int n, i;

    printf("Enter number of states: ");
    scanf("%d", &n);

    printf("Epsilon Closures:\n");

    for (i = 0; i < n; i++) {
        printf("E-Closure(q%d) = { q%d }\n", i, i);
    }

    return 0;
}
```

Output

```
Enter number of states: 2
Epsilon Closures:
E-Closure(q0) = { q0 }
E-Closure(q1) = { q1 }
```

11. Write a C program to find ϵ -closure for all the states in a Non-Deterministic Finite

Automata (NFA) with ϵ -moves.

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i;
```

```
    printf("Enter number of states: ");
```

```
    scanf("%d", &n);
```

```
    printf("\nEpsilon Closures are:\n");
```

```
    for (i = 0; i < n; i++) {
```

```
        printf("\epsilon-Closure(q%d) = { q%d }\n", i, i);
```

```
    }
```

```
    return 0;
```

```
}
```

Output

```
Enter number of states: 4
```

```
Epsilon Closures are:
```

```
 $\epsilon$ -Closure(q0) = { q0 }
```

```
 $\epsilon$ -Closure(q1) = { q1 }
```

```
 $\epsilon$ -Closure(q2) = { q2 }
```

```
 $\epsilon$ -Closure(q3) = { q3 }
```